

Laboratorio

Le architetture funzionano con la logica binaria (1-0, on-off, 0V-3V). La logica acceso spento (rimuovendo i valori intermedi), riduce notevolmente la possibilità di commettere errori nel determinare la soglia.

Sistema di numerazione posizionale: esempio, quello decimale

10 simboli {0,1,2,3,4,5,6,7,8,9}. Esempio 102 centinaia, decine, unità

Possiamo rappresentare qualunque quantità, è un sistema flessibile.

Sistema di numerazione additivo: esempio, quello romano (PS, anche in questo caso la posizione conta → VI è diverso da IV).

Al crescere della quantità servono però nuovi simboli, il sistema additivo ci consente di rappresentare infiniti numeri senza avere infiniti simboli.

Nei sistemi di numerazioni posizionali abbiamo quindi la base (binaria, ottale, esadecimale) e i simboli da usare per rappresentare i numeri.

$\sum_{i=0}^{n-1} (p(i) \cdot B^i)$ Esempio: $120 \rightarrow 1 \cdot 10^2 + 2 \cdot 10^1 + 0 \cdot 10^0$ (rappresentazione tramite

somma di potenze della base → convertiamo da qualsiasi base alla base decimale)

Per la base sedici aggiungiamo le lettere da A a F

$$126_{16} = 1 \cdot 16^2 + 1 \cdot 16^1 + 6 \cdot 16^0$$

$$(ABC)_{16} = A \cdot 16^2 + B \cdot 16^1 + C \cdot 16^0 = 10 \cdot 16^2 + 11 \cdot 16^1 + 12 \cdot 16^0$$

Per la base otto uso simboli da 0 a 7

$$(126)_8 = 1 \cdot 8^2 + 1 \cdot 8^1 + 1 \cdot 8^0$$

$$(121)_8 = 1 \cdot 8^2 + 2 \cdot 8^1 + 1 \cdot 8^0$$

Per la base binaria uso 0 e 1

126 non lo posso rappresentare come somma di potenze di base in base 2, lo dovrei convertire. 101 lo posso rappresentare:

$$(101)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

Per convertire da base 10 a base b usiamo la tecnica delle divisioni successive (divido per la base e tengo conto del resto finché non arrivo a zero).

Con i numeri interi in binario il range è $[0, 2^3)$ → significa che 0 è incluso, 8 è escluso

Esempi con basi astruse:

Da base 10 a base 8 ($65_{10} = 101_8$):

Div : $65 \rightarrow 8 \rightarrow 1 \rightarrow 0$

Resto $1 \rightarrow 0 \rightarrow 1$

Altro esempio con $67_{10} = 103_8$

Div : $67 \rightarrow 8 \rightarrow 1 \rightarrow 0$

Resto $3 \rightarrow 0 \rightarrow 1 \rightarrow \text{stop}$

Da base 10 a base 16:

$4091_{10} = FFB_{16}$

Div : $4091 \rightarrow 255 \rightarrow 15 \rightarrow 0$

Resto $11 \rightarrow 15 \rightarrow 15 \rightarrow \text{stop}$

$479_{10} = 1DF_{16}$

Div : $479 \rightarrow 29 \rightarrow 1 \rightarrow 0$

Resto $15 \rightarrow 13 \rightarrow 1 \rightarrow \text{stop}$

Numeri interi con segno

- 1) segno e valore assoluto→il primo bit indica il segno, gli altri restano invariati
se $3 = 011_2$ allora $-3 = 111_2$
- 2) complemento a uno→invertiamo tutti i bit
se $3 = 011_2$ allora $-3 = 100_{c1}$
- 3) complemento a due→invertiamo tutti i bit e sommiamo uno
se $3 = 011_2$ allora $-3 = 100_{c1} + 001 = 101_{c2}$
se $2 = 010_2$ allora $-2 = 101_{c1} + 001 = 110_{c2}$ **Perchè c'è il riporto, il terzo bit diventa zero**

Per i numeri **positivi**, **il risultato non cambia in base alla tecnica**; per quelli negativi cambia in base alla tecnica. Ricordiamo che il most significant bit è zero se il numero è positivo, 1 se è negativo.

Quando si verifica un trabocco in una somma:

- i due addendi hanno stesso segno
- e il bit di segno di somma è diverso da quelli degli addendi

Le somme si fanno in colonna, il riporto:

- per i numeri interi è sempre overflow
- per i numeri relativi non sempre

Per le differenze, tipo $A-B$, devo fare la somma e convertire B con complemento a due:

$A+(-B)$

0 0 1 0 +

0 0 1 1 =

0 1 0 1

Assembly

Il calcolatore esegue istruzioni sequenzialmente.

Compilatore: traduce in codice assembly o, linguaggio macchina assembly.

L'assemblatore poi traduce in linguaggio binario.

La memoria è formata da un insieme di parole, la dimensione delle word nella memoria è ben definita (32, o 64 bit). Inoltre ogni parola è associata ad un indirizzo ben specifico.

Se abbiamo 32 bit per Word, gli indirizzi aumenteranno di un termine 4 nel programma.

Big endian→ in maniera crescente, i bit nell'indirizzo vengono scritti da sinistra verso destra (0,1,2,3)

Little endian→in maniera decrescente, da destra verso sinistra. (3,2,1,0; 7,6,4,3)

Risc→isa ridotto, ogni istruzione occupa solo una Word in memoria.

Cisc→CISC, più complesse, ogni istruzione può occupare più word di memoria

La memoria nell'architettura risc viene usata per contenere sia dati che istruzioni.

Ci sono nell'architettura risc 16 registri, usati in questo modo:

- R0 non si usa in genere
- R1-R12→normale uso, metto i numeri, i puntatori
- R13→SP, Stack Pointer
- R14→Link Register (mantiene in memoria l'indirizzo di ritorno quando viene chiamata una routine)

- R15→PC, Program Counter (tiene l'indirizzo in memoria della prossima istruzione da eseguire)

Metodi di indirizzamento:

- Modo di **registro**→ADD R1,R2,R3 (usiamo tre modi di registro)
- Modo assoluto/diretto→usato nell'istruzione LOAD R2, NUM1 (LOC)
- Modo **immediato**→ADD R4,R6,#200 (l'operando è dato nell'istruzione)
- Modo **indiretto**→LOAD R2, [R5]
- Modo **con indice e spiazzamento**→LOAD R2, [20,#5] All'indirizzo dentro il registro TT sommo 20.

Ci sono poi anche le direttive di assemblatore, servono per dargli comandi specifici, o informazioni aggiuntive utili:

- Dataword, per caricare i dati in memoria; in Visual è DCD
- Costante, con EQU→ Nome EQU valore_numerico
- Origin→ Serve per indicare l'indirizzo di inizio delle istruzioni, in visual non c'è
- Reserve→per riservare memoria, in Visual è FILL
- END→indica la fine del programma

Per indicare un commento uso il punto e virgola.

Struttura di una linea assembly:

Etichetta (facoltativo), Operazione, Operandi, commento

Visual2 implementa un'architettura arm con word da 32 bit, usiamo queste istruzioni: ldr, str, add, sub, mov. Tutti seguono la struttura destinazione, sorgente tranne per str che li inverte (sorgente-destinazione).

Per settare dati e costanti, dcd (dataword) e equ

Per riservare spazio per una variabile in memoria usiamo Etichetta, Fill 4 (o più bytes)

Istruzioni condizionali e cicli

Prima di fare un controllo logico devo impostare i bit di esito (o condizione), che sono NZCV. Negative, Zero, Carry (trabocco in binario naturale), Overflow (trabocco in complemento a due).

Posso farlo direttamente con adds e subs, che eseguono l'operazione cambiando anche i bit di esito, oppure usando **cmp** (fa la differenza tra i due registri) o **cmn** (fa la somma).

Queste ultime due istruzioni non modificano i registri, ma solo i bit di esito.

Dopo che ho aggiornato i bit di esito, posso fare il branch if, un goto con la condizione.

Sintassi: B+condizione etichetta

Casi possibili:

beq/bne→branch if equal/not equal

bgt/bge→branch if greater than/or equal

blt/ble→branch if less than/or equal

b→branch incondizionato

Pila

Lo stack è una struttura dati di tipo LIFO. Pensiamo ad una pila di piatti.

È sostanzialmente una lista di elementi.

Ha due operazioni, push e pop, quindi si può lavorare solo agendo sulla cima dello stack, il registro R13 (Stack Pointer) punta proprio alla cima dello stack.

Push (metti come primo elemento)→ STM[codice] SP! {lista dei registri}

Pop (togli il primo elemento e dammi il valore)→ LDM[codice] SP! {lista dei registri}

Codici, dal più usato al meno usato:

- **FD (Full descending)**, quando **metti** un **elemento** nello stack (push), **l'indirizzo** in R13 **diminuisce**.
- FA (Full ascending)
- ED (empty descending)
- EA (empty ascending)

Il punto esclamativo si mette sempre per dirgli di aggiornare il puntatore, lo stack classico usa come codice FD.

In assembly lo stack si usa per:

- passare più di 13 valori come argomenti ad una funzione
- mantenere lo storico di tutte le chiamate alle funzioni fatte per poter andare a ritroso (pensiamo alla ricorsione)
- salvare temporaneamente i 13 registri, per poterli modificare (magari dentro una routine) e poi poterli reimpostare (**più frequente**)

Sottoprogrammi

Quando viene chiamata una funzione, una parte dello stack viene riservata a quella funzione (stack frame, o area di attivazione). Lo stack frame contiene:

- i parametri passati alla funzione
- il vecchio frame pointer, per poter tornare al programma chiamante (o caller)

I sottoprogrammi (o routine) vengono utilizzati per organizzare meglio il codice, sono un insieme di istruzioni che possono essere richiamate in un qualsiasi momento durante l'esecuzione di un programma.

Il programma caller esegue un salto all'indirizzo della routine, poi quest'ultima dovrà eseguire un salto per continuare l'esecuzione del programma caller (per questo serve il Link Register). In VisuAL:

- Call→ bl indirizzo_Programma (salva il contenuto di PC in LR, e salta all'indirizzo specificato), **branch link** (collegamento ad una subroutine)
- Return→ mov PC, LR (torna alla prossima istruzione del programma chiamante)

Se vogliamo chiamare più routine una dentro l'altra (per esempio ricorsione), dobbiamo memorizzare man mano tutti gli indirizzi per tornare all'istruzione precedente: il singolo registro LR non basta e per questo motivo usiamo lo stack (tra l'altro, alcune architetture non hanno nemmeno il registro LR ma usano direttamente solo lo stack).

Per passare i parametri ad una funzione e ottenere poi il risultato della funzione:

- usiamo i 12 registri a disposizione (prima li salviamo nello stack, per poterli recuperare dopo)
- li passiamo tramite lo stack

Istruzioni per lavorare con i bit (in complemento a due)

AND/ORR/EOR Rd, Ra, Rb per calcolare **and**, **or** o **xor** bit a bit di due numeri

operazioni shift:

- shift logico; sposta i bit riempiendo gli spazi vuoti con zero, sempre
- shift aritmetico, sposta i bit mantenendo il segno (=i bit persi sono messi ad uno)

Logical Shift Left LSL→0011 0101 << 1 = 0110 1010 (ricopio saltando un bit, e aggiungo zero alla fine)

Logical Shift Right LSR→1011 0100 >> 2 = 0010 1101 (perdiamo l'informazione del segno, aggiungo due zeri e ricopio fino ad arrivare a 8)

Arithmetic Shift Left (identico a LSL)→1110 0001 << 1 = 1100 0010

Arithmetic Shift Right (ASR) → 1110 0000 >> 2 = 11111000

Rotazione verso sinistra (in Visual non c'è) → 1011 → 0111

Rotazione verso destra (ROR) → 1011 → 1101

LDRB → Load Byte (legge un byte dalla ram e lo mette negli 8 bit meno significativi del registro, ponendo gli altri a 0)

STRB → Store Byte (gli 8 bit meno significativi del registro source nella locazione in memoria)

Moltiplicare per due: LSL (shift logico a sx), esempi: 0001 → 0010 → 0100 → 1000

Dividere per due: LSR (shift logico a dx), esempi: 1000 → 0100 → 0010 → 0001 → 0000 (per gestire anche i numeri negativi mi conviene usare lo shift aritmetico, ASR)

Vedere se un numero è pari: faccio **and** tra numero e 1, se il risultato è zero è pari (uso cmp)

<https://blog.wokwi.com/bitwise-operators-in-gifs/>

Algebra booleana

Ogni variabile assume solo 2 valori, 1 o 0 (segnale alto, 3.3V, segnale basso, 0V).

Quattro operazioni:

1) NOT → Negazione o Complemento $f(x_1) = \neg x_1 = \overline{x_1}$

2) AND → prodotto logico $f(x_1, x_2) = x_1 \cdot x_2 = x_1 \wedge x_2$

3) OR → somma logica $f(x_1, x_2) = x_1 + x_2 = x_1 \vee x_2$

4) XOR → differenza simmetrica (OR Esclusivo)

$$f(x_1, x_2) = (\neg x_1 \wedge x_2) \vee (x_1 \wedge \neg x_2) \quad \text{o} \quad x_1 \oplus x_2$$

Per le operazioni con due operandi valgono la proprietà commutativa e associativa.

Somma e prodotto hanno anche le loro identità, cioè *come si comportano con variabili 1 o 0?*

Somma: $1 + x = x$ $0 + x = x$

Prodotto: $1 \cdot x = x$ $0 \cdot x = 0$

Differenza simmetrica: $1 \oplus x = \neg x$ $0 \oplus x = x$

Per la negazione c'è la proprietà di involuzione (doppia negazione).

Precedenza operazioni: NOT, AND, OR, XOR

Teorema di De Morgan:

$$\neg(x_1 \vee x_2) = \neg x_1 \wedge \neg x_2 \qquad \neg(x_1 + x_2) = \overline{x_1} \cdot \overline{x_2}$$

$$\neg(x_1 \wedge x_2) = \neg x_1 \vee \neg x_2 \qquad \neg(x_1 \cdot x_2) = \overline{x_1} + \overline{x_2}$$

Le leggi di De Morgan (così come le varie proprietà distributive, complemento...) le dimostriamo con le tavole di verità.

Le tavole di verità ci consentono di verificare tutte le possibili interpretazioni di una funzione logica, funzioni con una o più variabili di ingresso e una variabile di uscita.

Ogni funzione ha una sua specifica tavola (una ed una sola).

Se due funzioni hanno la stessa tavola, allora sono equivalenti.

$f \rightarrow 1^\circ T$ (una funzione è associata **ad un'unica** tavola)

$f \leftarrow 1^\circ T$ (una tabella può essere associata a più funzioni, perchè ho applicato delle proprietà)

Una tabella ha 2^n righe (dove n è il numero di variabili).

Le espressioni logiche vengono scritte in genere in due forme standard (canoniche), SOP (=Sum of Products, DNF) e POS (=Prodotto di somme, CNF)

Procedimento per creare una SOP (=DNF) partendo da una tavola di verità:

- prendo tutti i casi (tutte le righe) dove il risultato è 1
- per ogni caso faccio il prodotto delle variabili booleane, se la variabile ha valore 0 prima va negata (i **mintermini**)
- sommo i prodotti

Esempio: $\overline{x_1}x_2 + x_1\overline{x_2}$

x_1	x_2	f_1
0	0	0
0	1	1
1	0	1
1	1	0

Procedimento per creare una POS (=CNF) partendo da una tavola di verità (è praticamente al contrario):

- prendo tutti i casi (tutte le righe) dove il risultato è 0
- per ogni caso faccio la somma delle variabili booleane, se la variabile ha valore 1 prima va negata (i **maxtermini**)
- multiplico le somme

Esempio: $(x_1 + x_2) \cdot (\overline{x_1} + \overline{x_2})$

x_1	x_2	f_1
0	0	0
0	1	1
1	0	1
1	1	0

Se due espressioni logiche hanno la stessa tabella di verità, si dicono **equivalenti**.

Talvolta nelle tavole di verità che ci vengono fornite alcuni risultati della funzione logica non ci interessano, si ha quindi la don't care condition e possiamo decidere noi arbitrariamente i valori (conviene ovviamente scegliere quelli che semplificano i conti).

Costo e funzioni minimizzate

Quanto è onerosa da computare un'espressione?

Secondo il criterio di costo dei letterali, il costo di un'espressione è determinato da quante volte le variabili compaiono nell'espressione.

Il nostro obiettivo è semplificare l'espressione riducendo il costo al minimo e ottenendo così un'espressione in forma minima (e dunque una funzione minimizzata), ciò consente di semplificare il circuito logico riducendo costi di produzione e risparmiando corrente.

Per semplificare utilizzo alcune proprietà dell'algebra di boole

distributiva 1 $\rightarrow x + (y \cdot z) = (x + y) \cdot (x + z)$

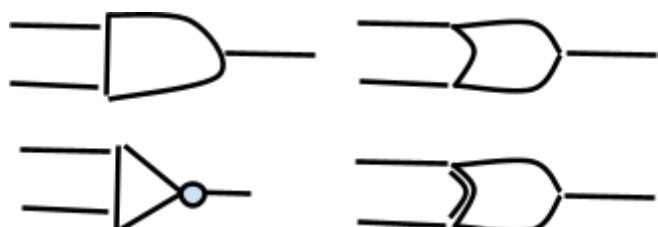
distributiva 2 $\rightarrow x \cdot (y + z) = (x \cdot y) + (x \cdot z)$

di idempotenza $\rightarrow x + x = x$ e $x \cdot x = x$

di complemento $\rightarrow x + \overline{x} = 1$ e $x \cdot \overline{x} = 0$

identità $\rightarrow 1 + x = 1$ e $0 \cdot x = 0$

Minimizzare una funzione logica è fondamentale per quando poi andiamo a costruire il **circuito combinatorio** (o **rete combinatoria**) inserendo le porte



logiche (una per ogni operatore) rispettando le priorità:

- AND (alto, sx)
- OR (alto, dx)
- NOT (basso, sx)
- XOR (basso, dx)

Per AND e OR possiamo avere delle porte logiche con più di due variabili come input, in questo caso sfruttiamo la proprietà associativa.

Le espressioni POS e SOP hanno due livelli:

- Un circuito POS ha un livello di OR seguito da un livello di AND.
- Un circuito SOP ha un livello di AND seguito da un livello di OR.

Nella pratica, si usano due porte logiche universali per realizzare i circuiti, la NAND($\uparrow$) e la NOR($\downarrow$). Sono porte logiche universali perchè consentono di realizzare tutti i circuiti, godono

della proprietà commutativa

ma non della proprietà

associativa (non possiamo

scomporre una porta NAND/NOR in porte NAND/NOR a cascata).

Esempio di porta NOT fatta con un NAND:

Per avere una porta AND basta negare una porta NAND.

Fare mappa di karnaugh

